



C프로그래밍 (CSE2003)

6. Repetition/Recursion

The *do..while* Loop

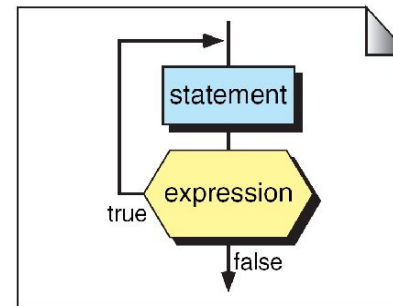
- **while** 문은 조건이 만족되지 않으면 루프를 한번도 실행하지 않는다.
- **do...while** 문은 반드시 한번은 루프를 실행하도록 되어있다.

□ Post-test Loop

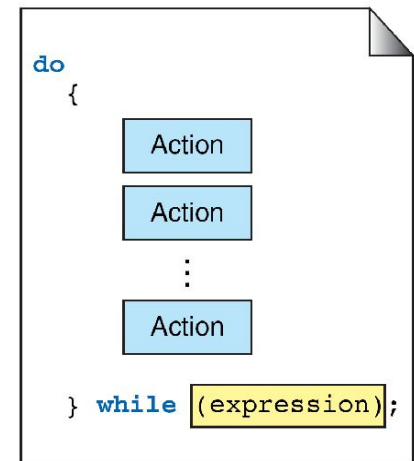
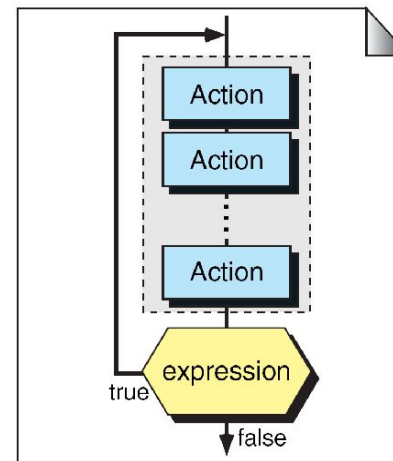
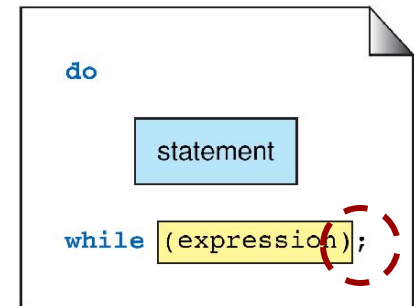
- 경우에 따라서는 **while** 문보다 **do...while** 문을 사용하는 것이

while(expression) 끝에 세미콜론(;)을 반드시 붙여야 한다!!!

Flowchart



Sample Code



The *do..while* Loop

예제 프로그램 – do..while loop

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int total = 0;
6     int val = 0;
7
8     do {
9         printf("Input number(Quit : 0) : ");
10        scanf("%d", &val);
11        total+=val;
12    }while(val != 0);
13
14    printf("Total : %d\n",total);
15    return 0;
16 }
```

사용자가 입력하는 수를 계속해서 더하는 프로그램

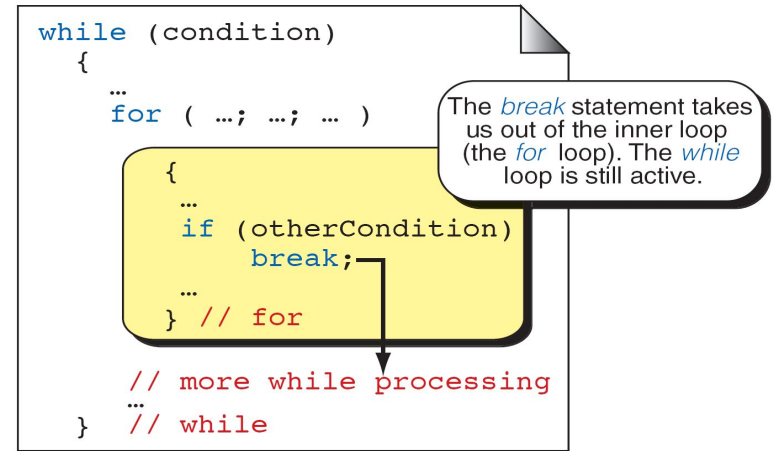
- 1) 먼저 사용자로부터 숫자를 입력받음
 - 2) 입력받은 값으로 진행할 것인지 말것인지 결정
- 최소한 한 번의 실행이 필요한 상황에서는
do ~ while 문이 while 문 보다 자연스럽다.

```
[root@mclab chap6]# vi chap6-4.c
[root@mclab chap6]# gcc -o chap6-4 chap6-4.c
[root@mclab chap6]# ./chap6-4
Input number(Quit : 0) : 2
Input number(Quit : 0) : 4
Input number(Quit : 0) : -5
Input number(Quit : 0) : 3
Input number(Quit : 0) : 1
Input number(Quit : 0) : 0
Total : 5
[root@mclab chap6]#
```

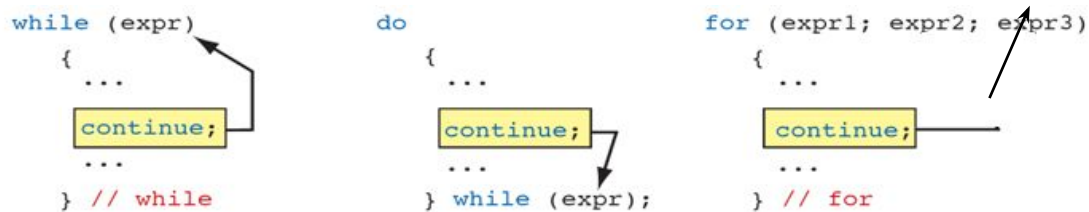
실습 24

break와 continue

- **break** : 가장 가까이서 감싸고 있는 반복문 하나를 빠져 나오게 한다.
주로 제한조건이 반복문, 조건문의 중간에 만족하였을 때 사용.



- **continue** : 돌던 루프의 남아있는 부분을 그냥 건너뛰는 것.
 - while이나 do-while문은 검사식(conditional expression)으로 제어 이동
 - for문은 변경식(update expression)으로 제어 이동



- **goto** : 구조적 프로그램에 적합하지 않으며 spaghetti code를 생산함.

break와 continue

예제 프로그램 - break와 continue

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int i, sum = 0;
6
7     for(i=0; ; i++) {
8         if(i>=100)
9             break;
10        else if(i%4 == 0)
11            continue;
12        sum+=i;
13    }
14    printf("sum=%d\n", sum);
15 }
```

1과 99사이의 4의 배수가 아닌 숫자들의
합을 구하는 프로그램

- 1) for 문에서 조건식이 없으므로 기본적으로는 무한루프를 돌게 되어 있다.
- 2) i가 100보다 커지는 경우 break를 만나 루프를 빠져나옴.
- 3) 4의 배수인 경우에는 continue를 만나서 sum에 더해지지 않는다.

```
[root@mclab chap6]# vi chap6-5.c
[root@mclab chap6]# gcc -o chap6-5 chap6-5.c
[root@mclab chap6]# ./chap6-5
sum=3750
[root@mclab chap6]#
```

실습 25

재귀호출(Recursion)

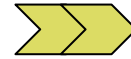
- 프로그램에서 동일한 작업을 반복하여 수행할 수 있는 방법은?
 - 앞서 배운 while, for, do..while 와 같은 반복문(loop) 사용. - (iterative method)
 - 또 다른 방법은 재귀호출(Recursion)을 이용하는 방법이 있다!!
- **Recursion 이란?**
 - 함수가 자기 자신을 반복하여 호출하는 형태.
 - 자기 스스로 반복 호출 함으로써, 코드를 간단/명료하게 만들 수 있다는 장점이 있다.
 - Limitation of Recursion.
 - 실제 속도가 iterative에 비해 떨어지므로 개념적으로만 사용한다.
 - 예외로 일부 특수한 문제는 recursion을 사용할 수밖에 없음.
 - 특수한 문제가 아니면 반복적 정의(while, for, do~while)로 처리 가능.

재귀호출(Recursion)

- Iterative function vs Recursive function

< iterative factorial definition >

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * (n-1) * (n-2) \dots 3 * 2 * 1 & \text{if } n > 0 \end{cases}$$



$$\text{factorial}(4) = 4 * 3 * 2 * 1 = 24$$

< recursive factorial definition >

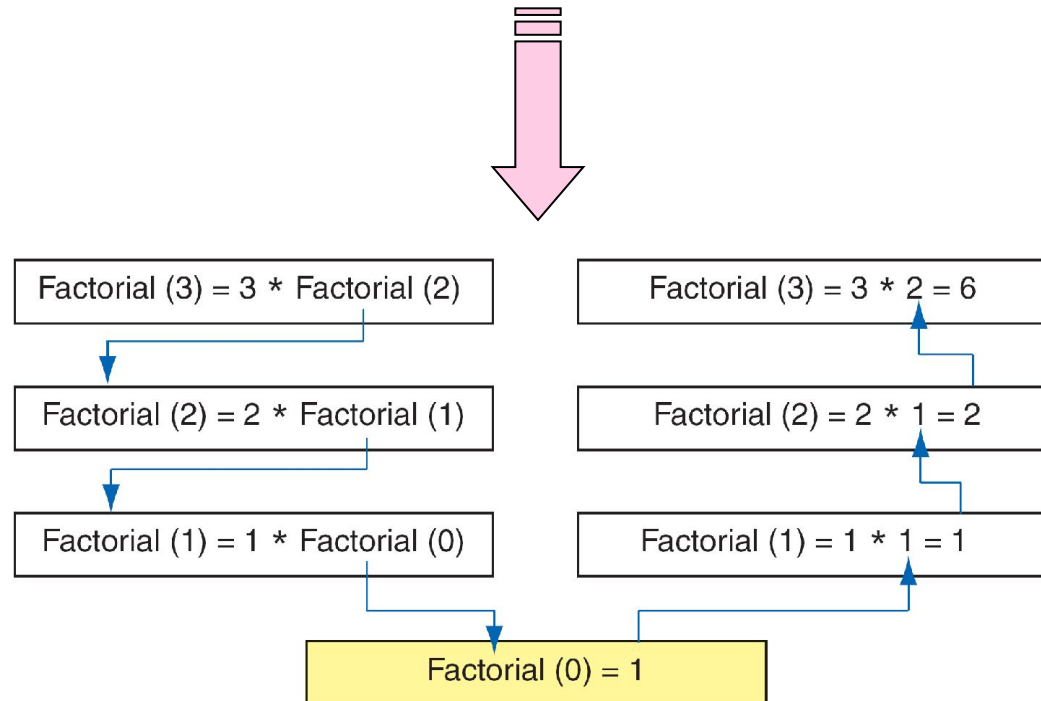
$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * \text{factorial}(n-1) & \text{if } n > 0 \end{cases}$$



$$\begin{aligned} \text{factorial}(4) &= 4 * \text{factorial}(3) \\ \text{factorial}(3) &= 3 * \text{factorial}(2) \\ \text{factorial}(2) &= 2 * \text{factorial}(1) \\ \text{factorial}(1) &= 1 * \text{factorial}(0) \end{aligned}$$

재귀호출(Recursion)

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * \text{factorial}(n - 1) & \text{if } n > 0 \end{cases}$$



< factorial(3) recursively >

Recursion solution(1/2)

예제 프로그램 - recursion

```
1 #include <stdio.h>
2
3 int factorial(int n);
4
5 int main(void) {
6     int a;
7     printf("Input a number : ");
8     scanf("%d", &a);
9
10    printf("%d! = %d\n", a, factorial(a));
11 }
12
13 int factorial(int n) {
14     if(n==0)
15         return 1;
16     else
17         return n * factorial(n-1);
18 }
```

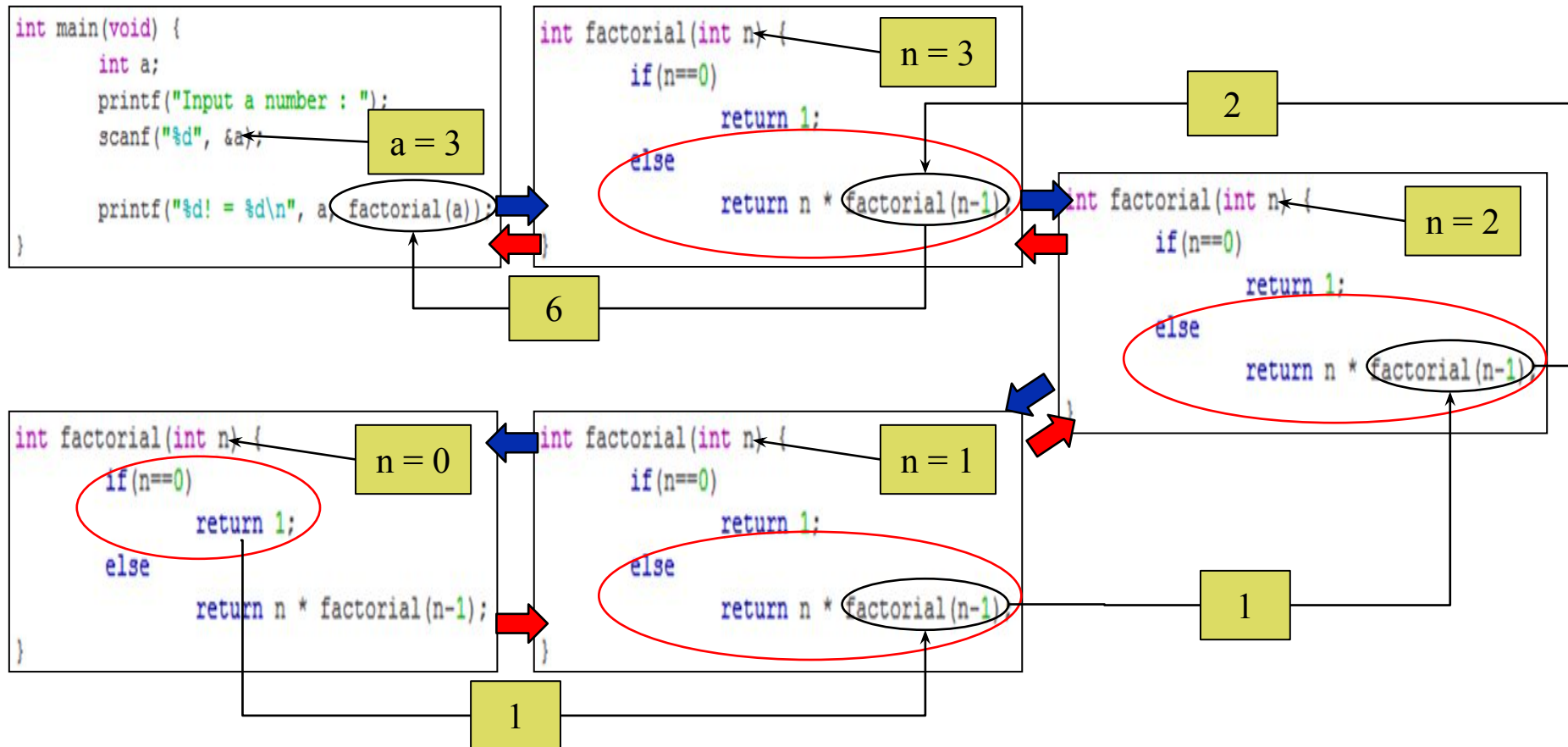
```
[root@mclab chap6]# vi chap6-6.c
[root@mclab chap6]# gcc -o chap6-6 chap6-6.c
[root@mclab chap6]# ./chap6-6
Input a number : 5
5! = 120
[root@mclab chap6]#
```

함수 factorial() 안에서 factorial()를
호출하는 프로그램

- 1) 호출할 때 인자는 매번 1씩 줄어드기
때문에 언젠가는 factorial(0) 을
호출하게 된다.
- 2) factorial(0)은 1을 반환한다.
- 3) 호출된 순서를 거꾸로 올라가며
factorial값을 구한다.

Recursion solution(2/2)

- a=3인 경우,



Iterative solution

- 예제 프로그램 - iterative

```
1 #include <stdio.h>
2
3 int factorial(int n);
4
5 int main(void){
6     int a;
7     printf("Input the number : ");
8     scanf("%d", &a);
9
10    printf("%d! = %d\n",a,factorial(a));
11    return 0;
12 }
13
14 int factorial(int n){
15     int factN = 1;
16     int i;
17     for(i = 1; i <= n; i++){
18         factN = factN * i;
19     }
20     return factN;
21 }
22
```

recursive function은
loop문을 이용하여
iterative로 도 구현할 수도 있다.

```
grl20100205@cspro:~$ gcc -o 6-7 6-7.c
grl20100205@cspro:~$ ./6-7
Input the number : 5
5! = 120
grl20100205@cspro:~$
```

Recursive 함수의 설계

- Designing Recursive Function

- 모든 recursive call은 problem의 size를 줄이거나, problem의 부분을 풀어야 한다.

- Recursive function을 design 하는 순서

1. base case 를 결정한다.

2. general case 를 결정한다.

3. base case와 general case 를 function에 모두 적용시킨다.

모든 recursive function은 base case를 지닌다. base case는 실질적으로 문제를 해결하는 부분이다. Factorial 의 경우

Base case는 factorial(0)이다.

general case를 제외한 함수의 나머지 부분을 말한다.

factorial 의 경우, general case는 $n * \text{factorial}(n-1)$ 이다.

Fibonacci series

• 예제 프로그램 - Fibonacci series

```
#include <stdio.h>

long fib (long);

int main (void)
{
    int seriesSize;
    int i;

    printf("This program prints a Fibonacci series.\n");
    printf("How many numbers do you want?");
    scanf("%d", &seriesSize);
    if (seriesSize < 2)
        seriesSize = 2;

    printf("First %d Fibonacci numbers: \n", seriesSize);
    for (i = 0; i < seriesSize; i++)
    {
        if (i % 5)
            printf(", %8ld", fib(i));
        else
            printf("\n%8ld", fib(i));
    }
    printf("\n");
    return 0;
}

long fib (long num)
{
    if (num == 0 || num == 1)
        return num;

    return (fib (num - 1) + fib (num-2));
}
```

This program prints a Fibonacci series.
How many numbers do you want?30
First 30 Fibonacci numbers:

0,	1,	1,	2,	3
5,	8,	13,	21,	34
55,	89,	144,	233,	377
610,	987,	1597,	2584,	4181
6765,	10946,	17711,	28657,	46368
75025,	121393,	196418,	317811,	514229

실습 26